Shortest Paths and Spanning Trees

A network can support different optimization questions. The objective determines which algorithm to use.

Shortest paths: minimize the cost of traveling from a source to a destination.

Minimum spanning tree: connect every node while minimizing the sum of selected edge costs.

- Dijkstra: nonnegative travel costs
- Bellman–Ford: negative travel costs
- Prim and Kruskal: connecting all nodes
- Proofs and further explanation

Recall the network model

Term	Meaning
Node	A location or state
Directed arc (i,j)	Travel is allowed from i to j
Undirected edge {i,j}	Travel is allowed in both directions
Path	A sequence of adjacent nodes with no repeated node
Path cost	The sum of costs of its arcs or edges

For shortest paths we pay for one route. For a spanning tree we pay for every selected edge once.

A shortest path is a one-unit flow problem

Let E be the directed-arc set and $x_{ij} \geq 0$ the flow on arc (i, j), with cost c_{ij}. Send one unit from source s to destination t:

$$\begin{aligned} \min \quad & \sum_{(i,j)\in E} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j:(i,j)\in E} x_{ij} - \sum_{j:(j,i)\in E} x_{ji} = \begin{cases} 1 & i = s, \\ -1 & i = t, \\ 0 & \text{otherwise}, \end{cases} \\ & x_{ij} \geq 0. \end{aligned}$$

> With a reachable destination and no negative-cost directed cycle, a shortest path gives an optimal flow. Other optimal flows may split across equally short paths.

Represent an undirected edge by two opposite arcs with the same cost.

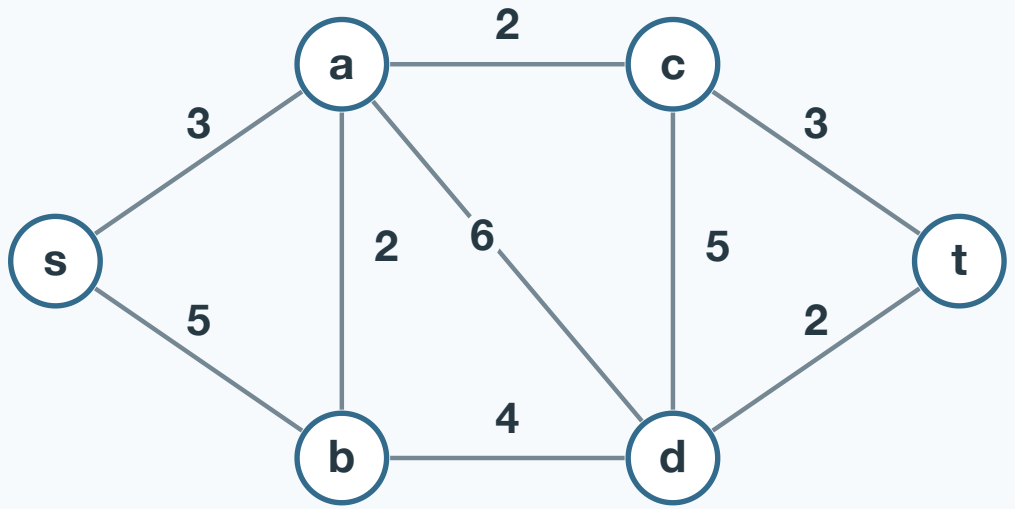
Find the shortest routes starting at s

Every edge can be traversed in either direction. Its number is the travel cost.

> Find the minimum cost from s to every node, especially t.

All nine edge costs are nonnegative. We will use Dijkstra's algorithm.

When distance labels tie, select nodes in this fixed order: s, a, b, c, d, t.



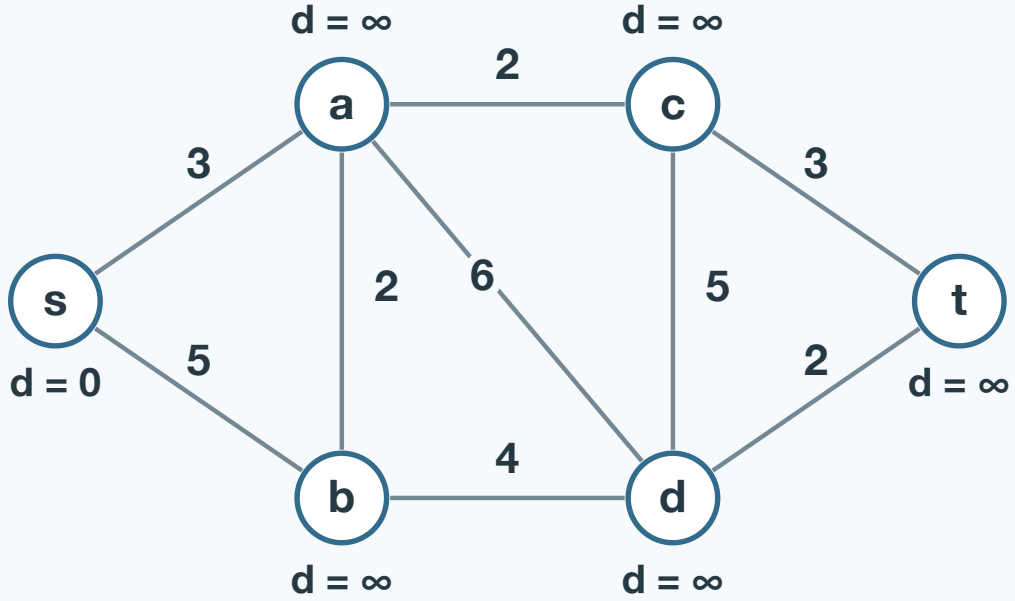
Full undirected network. The source is s; the destination of particular interest is t.

A distance label records the best route found so far

Write $d(v)$ for the cost of the best route found from s to node v. Initially $d(s) = 0$; all other labels are ∞.

> Along an arc or edge from u to v, compare $d(v)$ with $d(u) + c_{uv}$. If the new value is smaller, replace $d(v)$ and record u as the predecessor of v.

This comparison is called **relaxing** the arc or edge. A predecessor records the last step of the improved route.



Undirected edges; labels are costs. Filled nodes have final distances. Bold edges record current predecessors.

Dijkstra: finalize the smallest temporary label

1. Initialize the labels; no node is final yet.
2. Choose an unfinalized node u with the smallest finite label.
3. Mark u final. Relax each edge from u to an unfinalized neighbor.
4. Repeat. Stop when every reachable node is final.

> Keep the existing predecessor when a candidate route has the same cost. This fixes a reproducible choice among equally short routes.

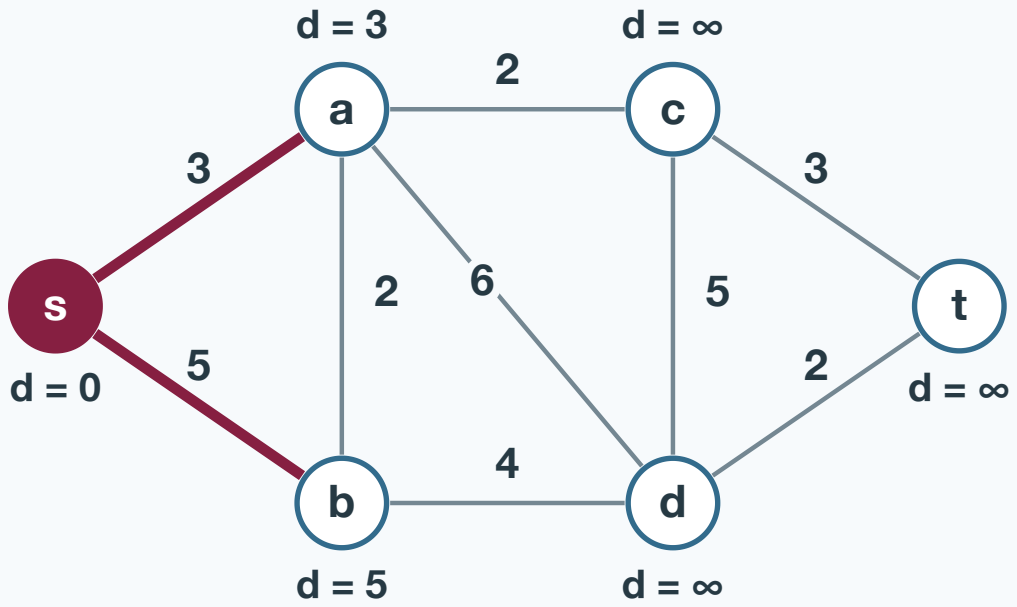
If only one destination matters, stop when that destination becomes final. A label first assigned to a node is not necessarily its final distance.

Select s: discover the first two routes

$$d(a) = 0 + 3 = 3, \qquad d(b) = 0 + 5 = 5.$$

Make s final. Nodes c, d and t have not yet been reached.

Predecessors: a ← s and b ← s. Next choose a because 3 is the smallest temporary label.



Undirected edges; labels are costs. Filled nodes have final distances. Bold edges record current predecessors.

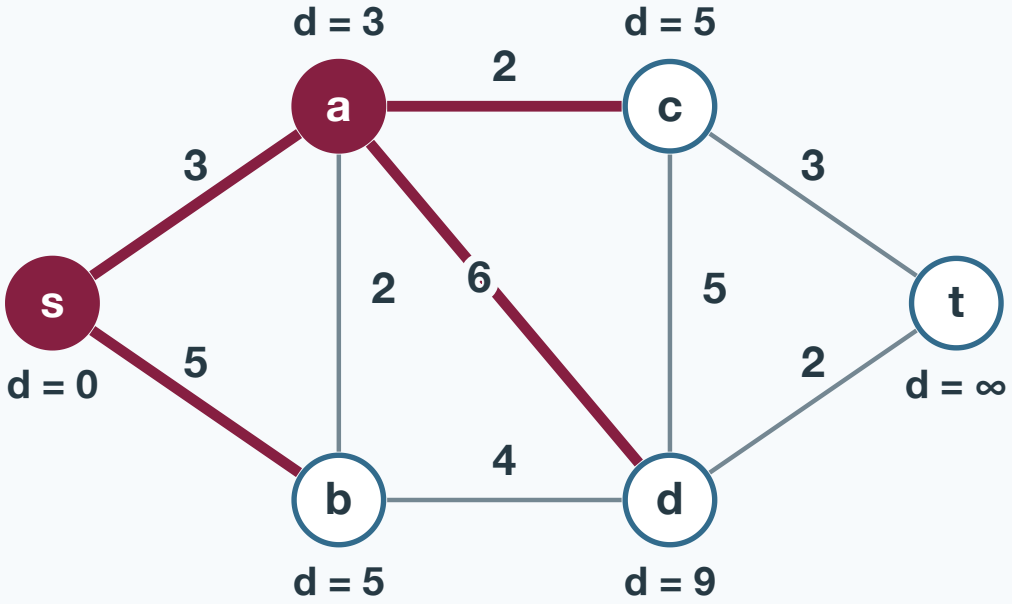
Select a: improve the routes to c and d

$$d(c) = 3 + 2 = 5, \qquad d(d) = 3 + 6 = 9.$$

The route through a to b costs $3 + 2 = 5$, equal to its current label. Keep predecessor s for b.

> Now b and c both have label 5. The stated node order chooses b next.

The label 9 at d is temporary; the algorithm will compare later routes to d.



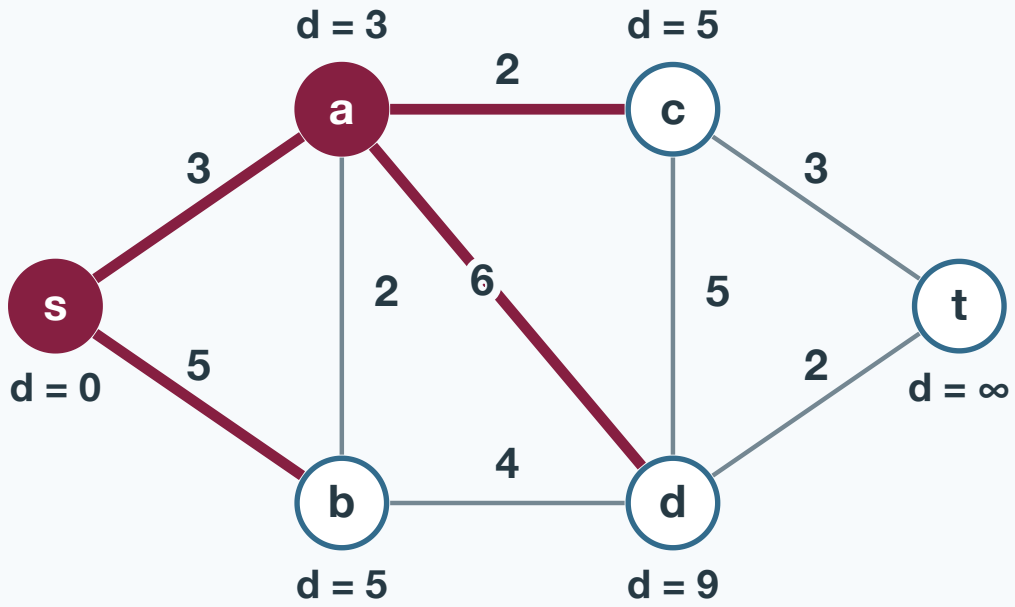
Undirected edges; labels are costs. Filled nodes have final distances. Bold edges record current predecessors.

Your turn: finish the distance table

Work for about 6 minutes. Node a has just become final; the current labels and predecessors are shown.

1. Select the next node and update its neighbors.
2. Continue until every node is final.
3. Follow predecessors backward from t to recover a shortest route.

Use node order s, a, b, c, d, t for ties. Keep the old predecessor when a route only ties the old cost.



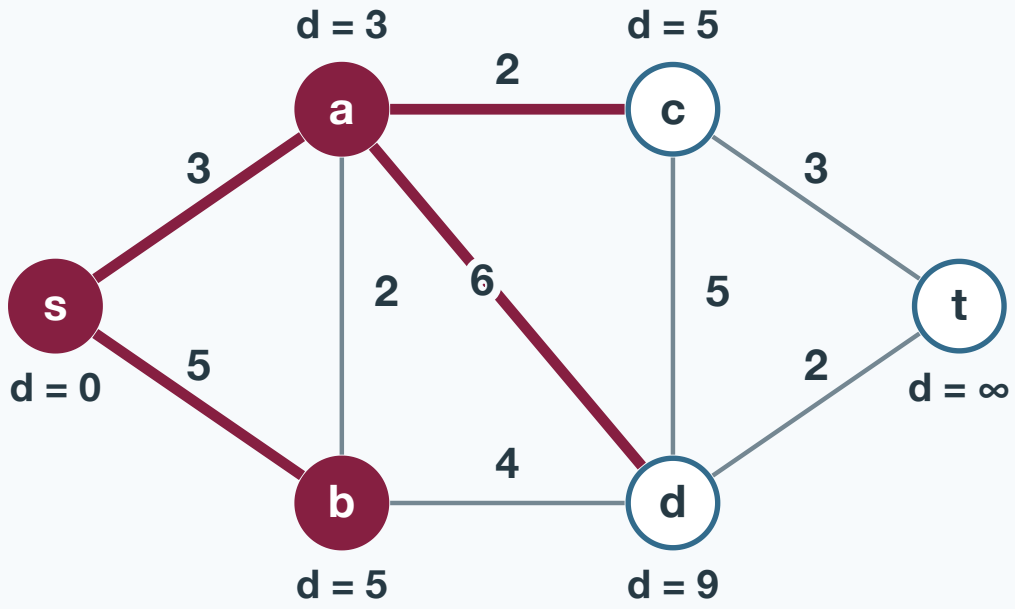
Undirected edges; labels are costs. Filled nodes have final distances. Bold edges record current predecessors.

Select b: no label improves

Node b has label 5. The candidate route to d costs $5 + 4 = 9$, equal to its existing label.

> Keep predecessor a for d. No label changes.

Node a is already final. Among the remaining nodes, c has the smallest label: 5.



Undirected edges; labels are costs. Filled nodes have final distances. Bold edges record current predecessors.

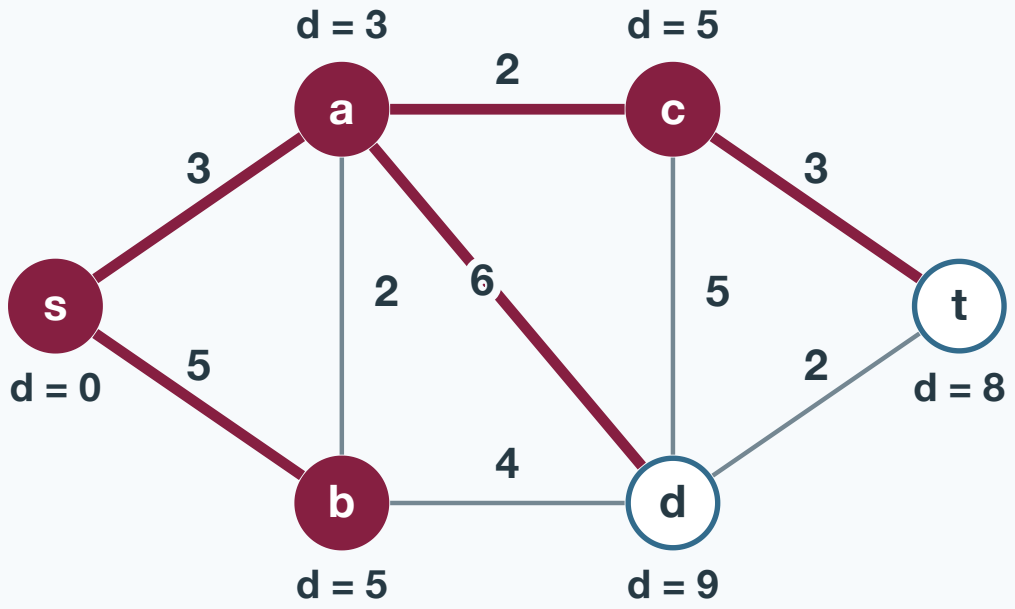
Select c: reach t with cost 8

$$d(t) = 5 + 3 = 8.$$

The candidate route to d costs $5 + 5 = 10 > 9$, so retain $d(d) = 9$.

> Next select t, with label 8, before d, with label 9.

The newly found route to t is s → a → c → t.



Undirected edges; labels are costs. Filled nodes have final distances. Bold edges record current predecessors.

Select t, then d: the full trace is complete

Selected	s	a	b	c	d	t
Initial	0	∞	∞	∞	∞	∞
s	0	3	5	∞	∞	∞
a	0	3	5	5	9	∞
b	0	3	5	5	9	∞
c	0	3	5	5	9	8
t	0	3	5	5	9	8
d	0	3	5	5	9	8

From t, the candidate route to d costs $8 + 2 = 10 > 9$. Selecting d afterward produces no further change.

> Final distances from s: (0, 3, 5, 5, 9, 8), in node order s, a, b, c, d, t.

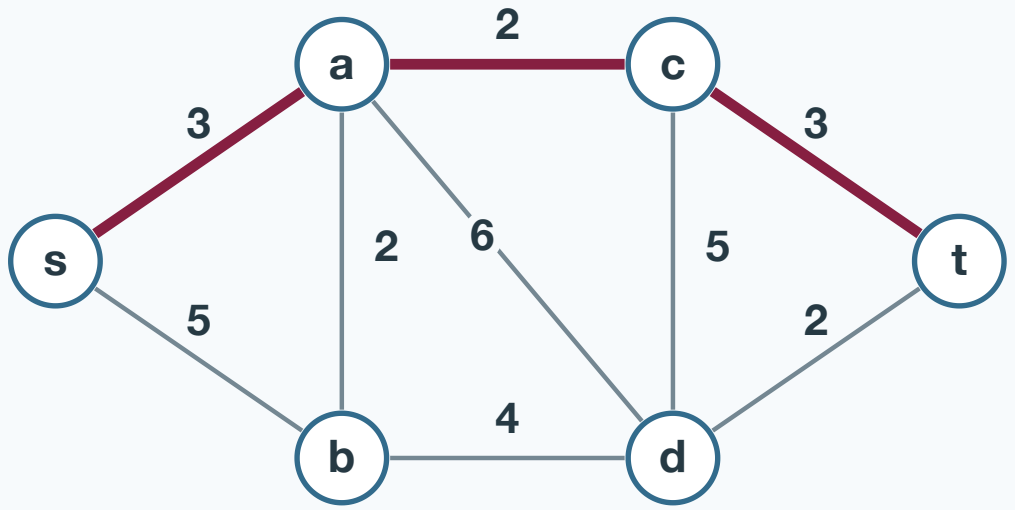
Recover the shortest path using predecessors

Start at t and read its predecessor c, then a, then s. Reverse this sequence.

$$s \to a \to c \to t, \qquad 3 + 2 + 3 = 8.$$

The route s → b → d → t costs 5 + 4 + 2 = 11, so it is longer.

> The highlighted route reaches t. The other predecessor edges still give shortest routes to the remaining nodes.



Bold edges form a shortest s-to-t route, with total cost 8.

Why nonnegative costs justify finalizing a node

> Suppose u has the smallest temporary label. Any undiscovered competing route must first reach some unfinalized node.

That first unfinalized node already has a label no larger than the cost of the competing route’s prefix. Its label cannot be smaller than the label selected at u.

> If every remaining edge cost is nonnegative, continuing along that route cannot lower its cost below the selected label at u.

Appendix: the complete argument. A negative edge can break exactly the “continuing cannot lower cost” step.

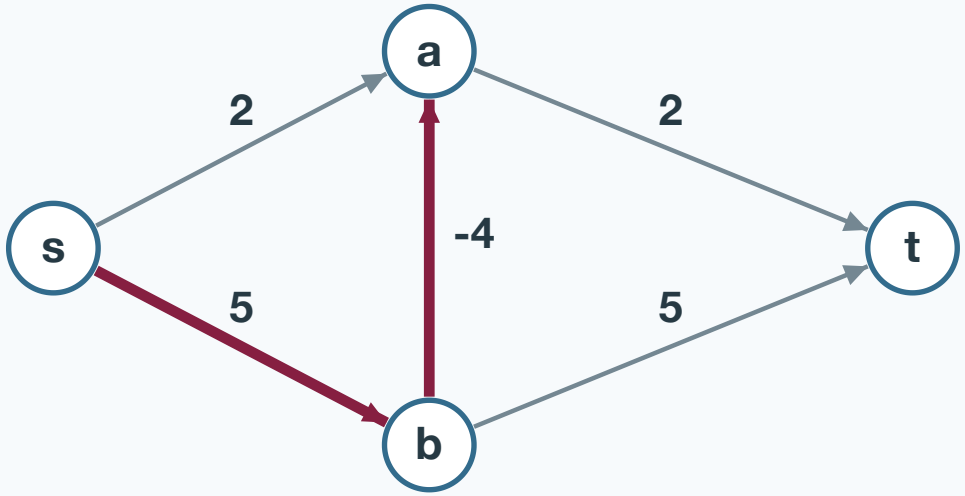
A negative arc can invalidate a final label

Arcs are now directed. From s, Dijkstra would label a as 2 and b as 5, then make a final.

$$s \to b \to a: \quad 5 + (-4) = 1 < 2.$$

> The later route through b improves a. Finalizing a at 2 was incorrect. This graph has no directed cycle.

We need an algorithm that allows later improvements instead of finalizing nodes.



Directed graph. The highlighted route to a costs 1; the direct arc costs 2.

Bellman–Ford: allow successively more arcs

Let $D_k(v)$ be the minimum cost of a **walk** from s to v using at most k arcs. A walk may repeat nodes. Set $D_0(s) = 0$ and $D_0(v) = \infty$ for $v \neq s$.

$$D_k(v) = \min\left\{ D_{k-1}(v),\quad \min_{(u,v)\in E}\left(D_{k-1}(u) + c_{uv} \right) \right\}.$$

> For each new row, use only the previous row's values. Either keep the old route, or append one incoming arc.

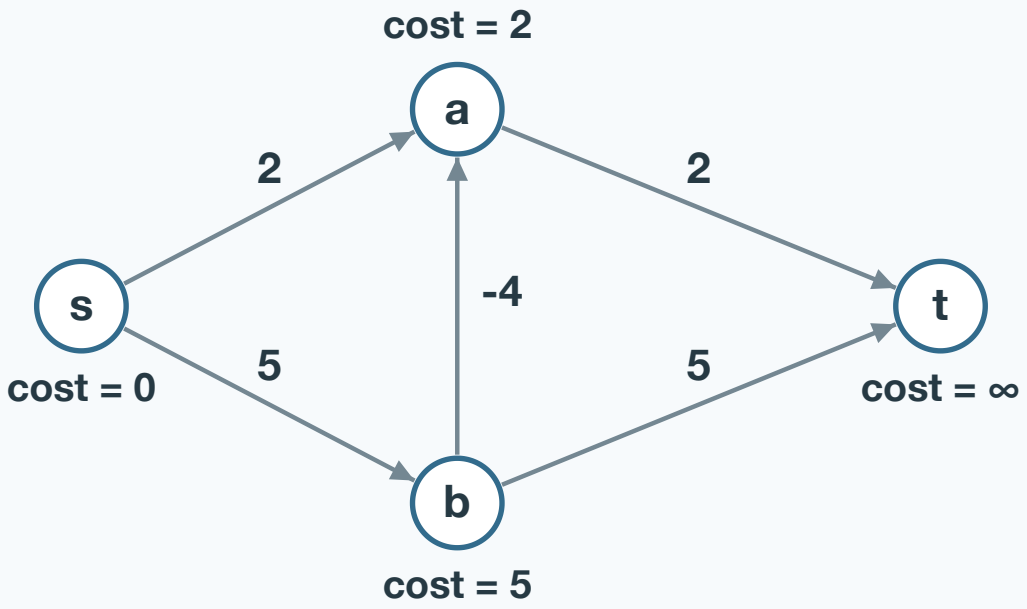
This is the synchronous version of Bellman–Ford: all nodes are updated together from a saved preceding row. Negative arc costs are allowed.

Pass 1: use at most one arc

Pass k	s	a	b	t
0	0	∞	∞	∞
1	0	2	5	∞

The one-arc routes from s reach a at cost 2 and b at cost 5. No one-arc route reaches t.

> Do not use the newly computed value at a to update t in this same pass. Every calculation uses row 0.



Directed arcs; arrowheads show allowed travel. Labels beside nodes are distances using at most the current number of arcs.

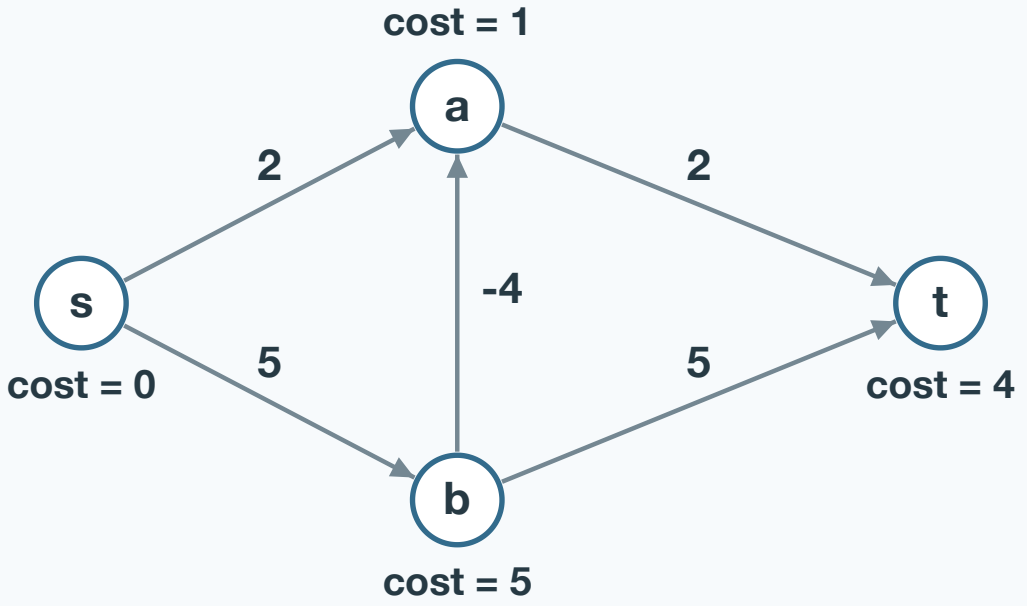
Pass 2: the route to a improves

Pass k	s	a	b	t
1	0	2	5	∞
2	0	1	5	4

$$D_2(a) = \min\{2, 5 - 4\} = 1.$$

$$D_2(t) = \min\{\infty, 2 + 2, 5 + 5\} = 4.$$

> The value 1 at a is in row 2. Use it for t only in the next pass.



Directed arcs; arrowheads show allowed travel. Labels beside nodes are distances using at most the current number of arcs.

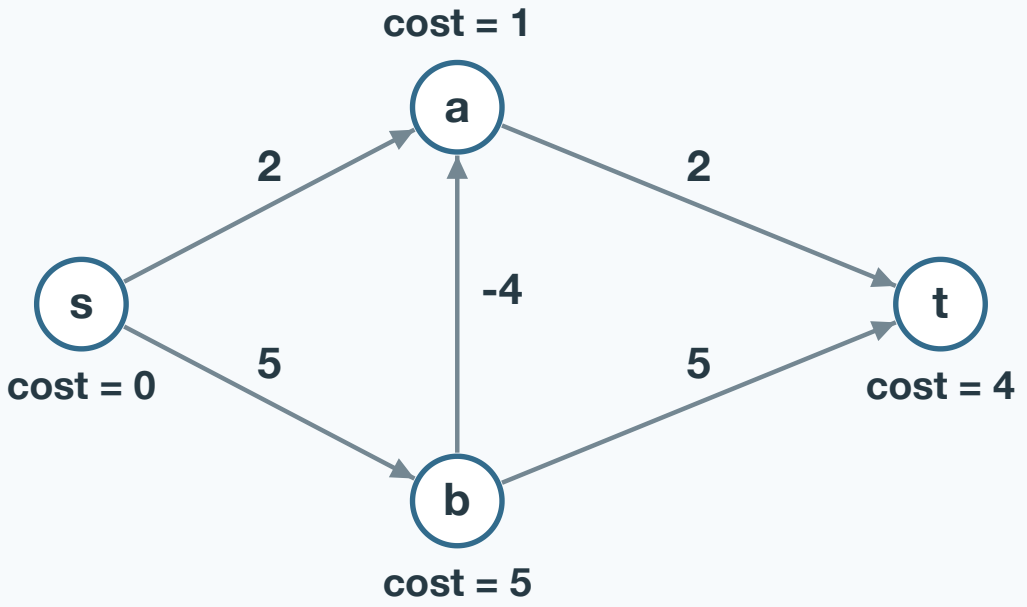
Your turn: complete pass 3

Work for about 4 minutes. Copy row 2 and compute row 3 using only row 2.

Pass k	s	a	b	t
2	0	1	5	4

1. Does the label at a change?
2. Which incoming arc gives the best new label at t?
3. Write the route that attains the new value.

Then check whether a fourth pass could improve any label.



Directed arcs; arrowheads show allowed travel. Labels beside nodes are distances using at most the current number of arcs.

Pass 3 gives the shortest route; pass 4 is unchanged

Pass k	s	a	b	t
0	0	∞	∞	∞
1	0	2	5	∞
2	0	1	5	4
3	0	1	5	3
4	0	1	5	3

$$D_3(t) = \min\{4, 1+2, 5+5\} = 3.$$

> The shortest route is s → b → a → t, with cost 5 – 4 + 2 = 3. There are four nodes, so a simple path uses at most three arcs.

The unchanged fourth row confirms that no further relaxation improves these distances.

When a finite shortest distance does not exist

A **negative-cost directed cycle** is a directed cycle whose arc costs sum to a negative number.

> If the cycle is reachable from s and can reach t, repeat the cycle before traveling to t. Each repetition lowers the cost, so there is no finite minimum walk cost to t.

For a graph with n nodes, a strict improvement on pass n detects a negative-cost cycle reachable from s. Only nodes reachable *from that cycle* have distance $-\infty$.

> An unreachable negative cycle does not affect distances from s. A reachable cycle that cannot reach t does not destroy the finite shortest distance to t.

Appendix: why the pass-n test works.

Choose the shortest-path algorithm by its assumptions

	Dijkstra	Bellman–Ford
Arc costs	All nonnegative	May be negative
Progress	Finalize a node	Allow one more arc per pass
Can a label improve later?	Not after it is final	Yes, on later passes
Typical work	$O(n^2 + m)$ by simple scans	$O(nm)$ for n passes
Negative-cycle information	Not applicable	Detects cycles reachable from s

Here n is the number of nodes and m the number of arcs; represent undirected edges by opposite arcs. A binary-heap Dijkstra implementation takes $O((n + m) \log n)$.

A spanning tree connects every node without a cycle

For an undirected graph, a **tree** is connected and has no cycle. A **spanning tree** uses every node of the original graph.

A tree on n nodes has exactly $n - 1$ edges. A connected graph has a spanning tree; a disconnected graph does not.

Tree operation	Consequence
Remove any tree edge	The nodes split into two connected components
Add an edge between two tree nodes	Exactly one cycle appears

These properties will also explain how a network-simplex basis changes. Today we minimize the total cost of the tree edges.

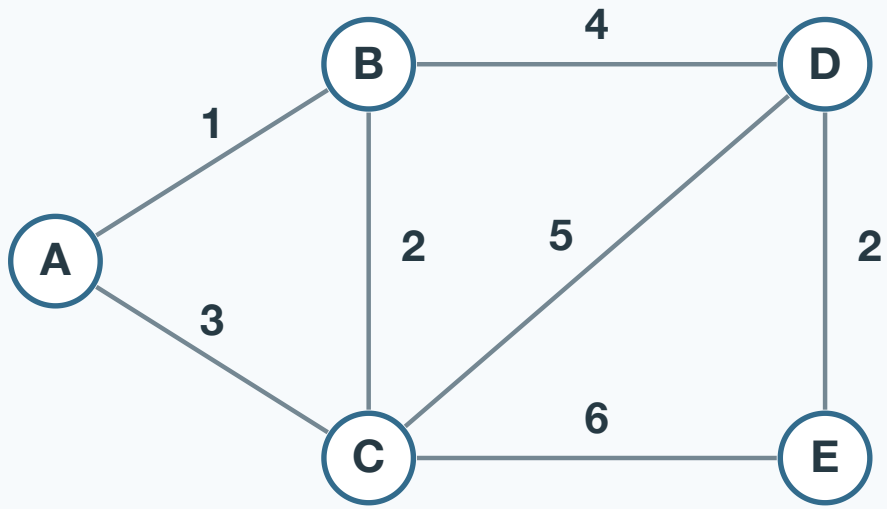
Choose four edges with the smallest total cost

The graph is undirected and connected. A spanning tree on these five nodes must contain four edges.

$$\min_{T \text{ a spanning tree}} \sum_{e \in T} c_e.$$

> We call this the **minimum spanning tree** problem, abbreviated MST.

Prim and Kruskal solve the same problem. For tied edge costs, compare endpoint pairs alphabetically: B–C precedes D–E.



Five nodes, seven candidate edges. Each edge cost is paid once when that edge is selected.

Prim: grow one connected tree

1. Start from one node; here start at A.
2. Consider every edge with one endpoint already reached and the other not yet reached.
3. Select a cheapest such edge and add its new node.
4. Repeat until every node has been reached.

> The reached and unreached node sets form a **cut**: a partition of the nodes into two groups. A crossing edge has one endpoint in each group.

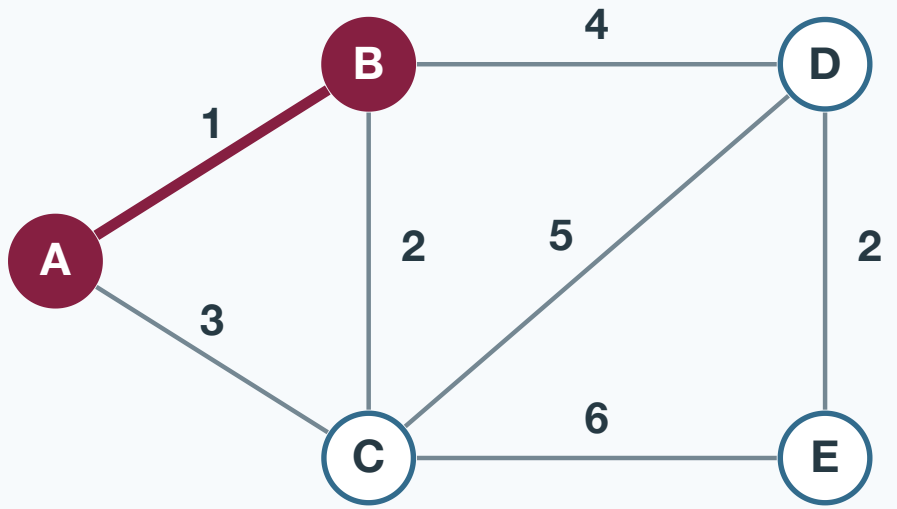
Prim chooses the cheapest edge crossing that cut. It compares single-edge costs, not distances from A.

Prim step 1: connect A to B

Crossing edge	Cost
A–B	1
A–C	3

> Choose A–B. Reached nodes: A, B. Total selected cost: 1.

The next candidates must connect {A,B} to {C,D,E}.



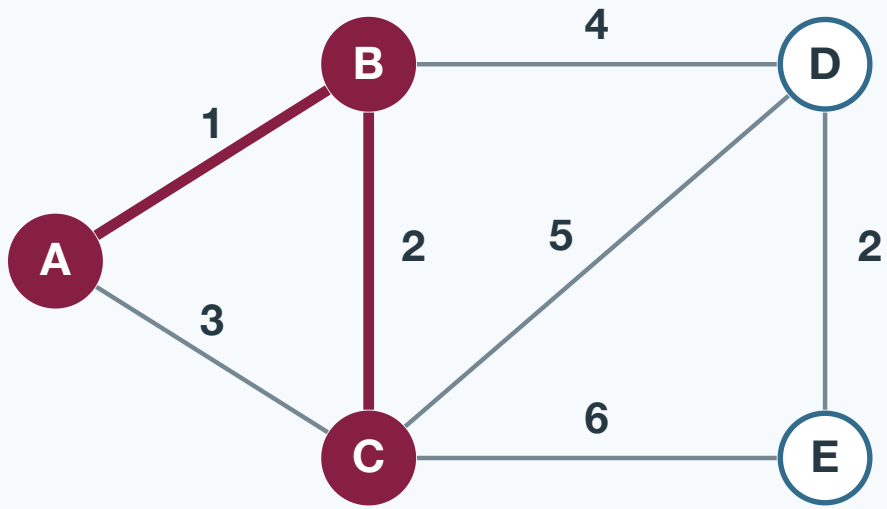
Undirected edges; labels are costs. Bold maroon edges have been selected.

Prim step 2: connect C through B

Crossing edge	Cost
A–C	3
B–C	2
B–D	4

> Choose B–C. Reached nodes: A, B, C. Total selected cost: 3.

Edge A–C now has both endpoints reached, so it is no longer a crossing edge.



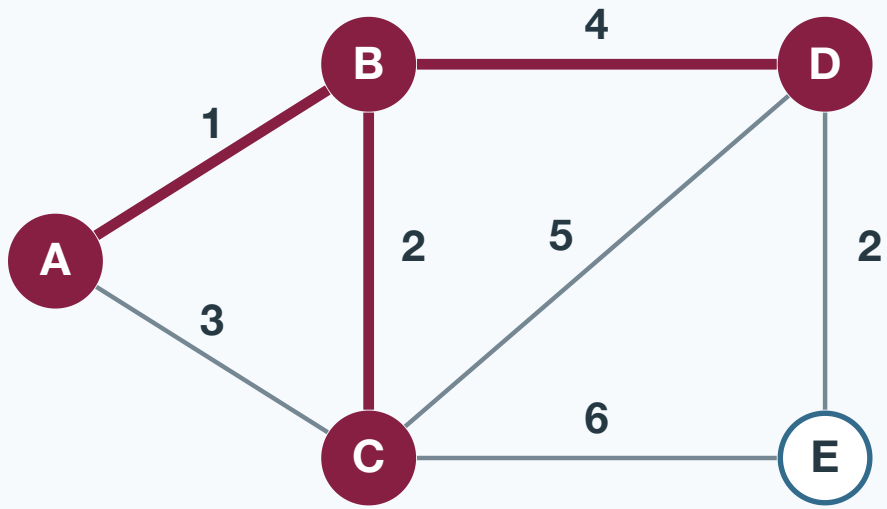
Undirected edges; labels are costs. Bold maroon edges have been selected.

Prim step 3: connect D through B

Crossing edge	Cost
B–D	4
C–D	5
C–E	6

> Choose B–D. Reached nodes: A, B, C, D. Total selected cost: 7.

Edge D–E has cost 2, but it could not be chosen earlier: both endpoints were outside the growing tree.



Undirected edges; labels are costs. Bold maroon edges have been selected.

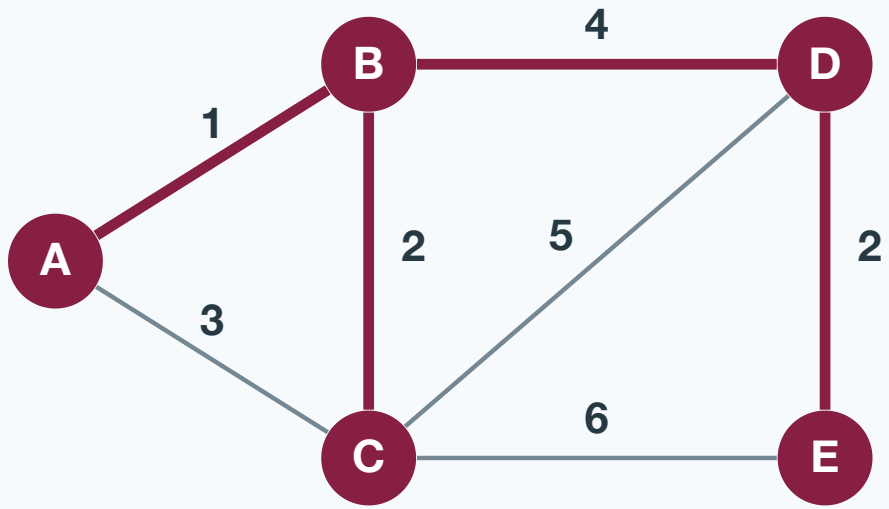
Prim step 4: connect E through D

Crossing edge	Cost
C–E	6
D–E	2

$$\boxed{1 + 2 + 4 + 2 = 9.}$$

All five nodes are connected by four edges. The selected tree has total cost 9.

Every step added one new node, so the selected edges never formed a cycle.



Undirected edges; labels are costs. Bold maroon edges have been selected.

Kruskal: add cheap edges without making a cycle

1. Sort every edge by increasing cost.
2. Start with no selected edges: each node is its own connected component.
3. Consider the next edge. Add it if its endpoints lie in different selected components; otherwise reject it.
4. Stop after selecting n – 1 edges.

> A **connected component** is a maximal group of nodes joined by paths using selected edges. Joining two different components cannot create a cycle.

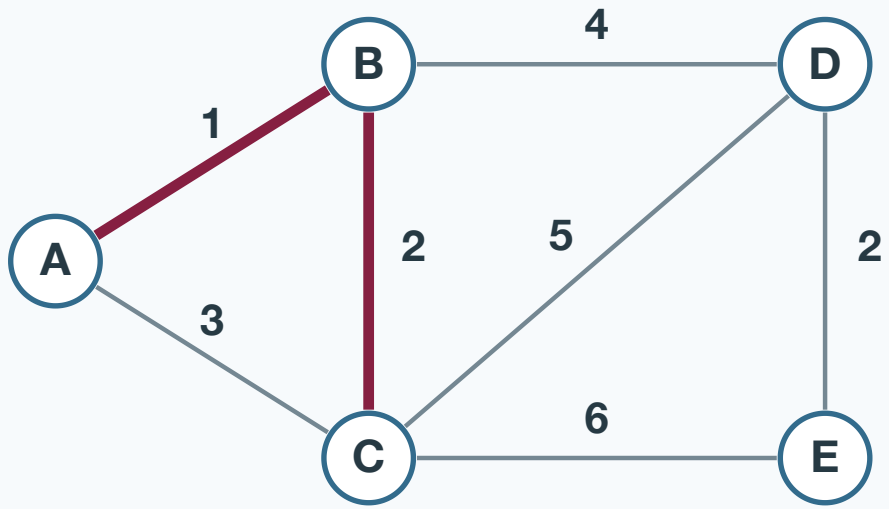
Unlike Prim, Kruskal may build several separate trees before joining them.

Kruskal steps 1–2: accept A–B, then B–C

Consider	Decision	Total cost
A–B, cost 1	Accept	1
B–C, cost 2	Accept	3

> Selected components are now {A,B,C}, {D}, and {E}.

The tied cost-2 edges are processed alphabetically, so B–C comes before D–E.



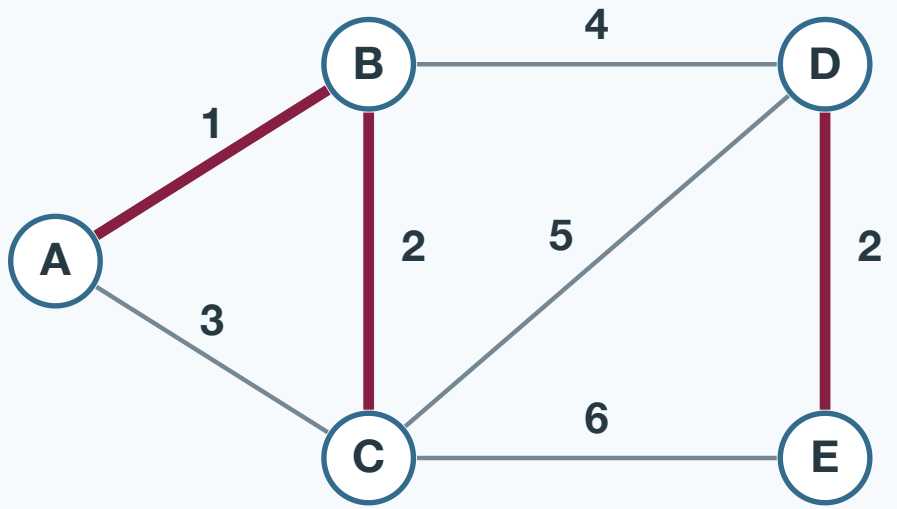
Undirected edges; labels are costs. Bold maroon edges have been selected.

Kruskal step 3: accept D–E

D and E lie in different selected components, so adding D–E creates no cycle.

> Components: {A,B,C} and {D,E}. Selected cost: 1 + 2 + 2 = 5.

The selected edges are not connected yet. Kruskal permits this intermediate state.



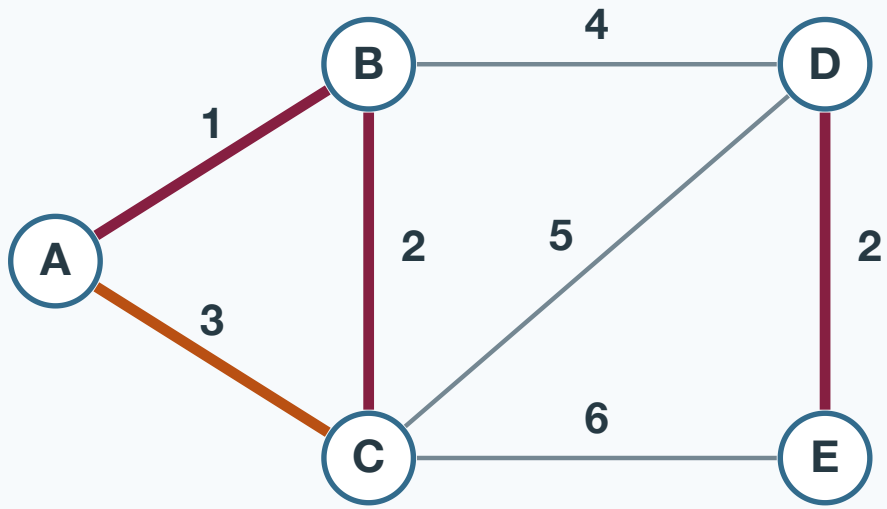
Undirected edges; labels are costs. Bold maroon edges have been selected.

Kruskal step 4: reject A–C

A and C are already connected by A–B–C. Adding A–C would create the cycle A–B–C–A.

> Reject A–C, even though its cost 3 is smaller than the next candidate cost 4.

Components and selected cost stay unchanged: {A,B,C}, {D,E}, and cost 5.

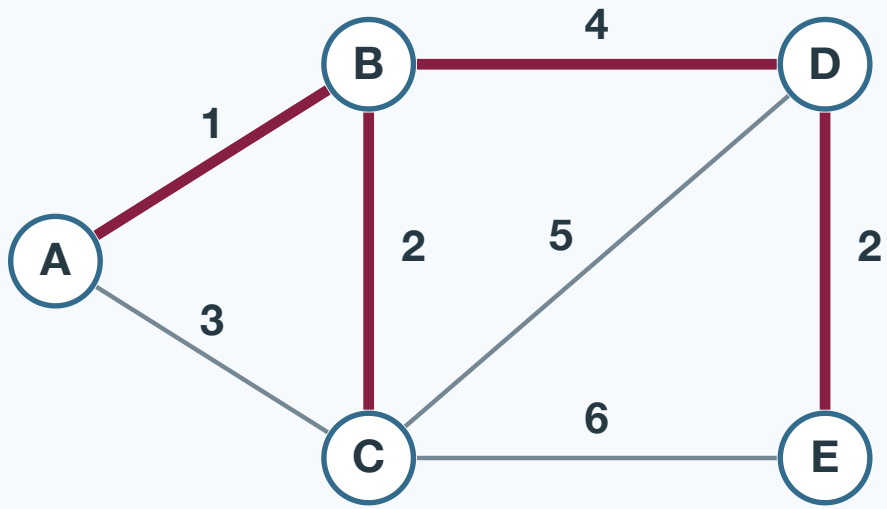


Orange A–C is being tested and rejected. Bold maroon edges remain selected.

Kruskal step 5: join the two components

Processed edge	Decision
A–B (1)	Accept
B–C (2)	Accept
D–E (2)	Accept
A–C (3)	Reject: cycle
B–D (4)	Accept

Four selected edges connect all five nodes. Stop with total cost 9, the same result as Prim.



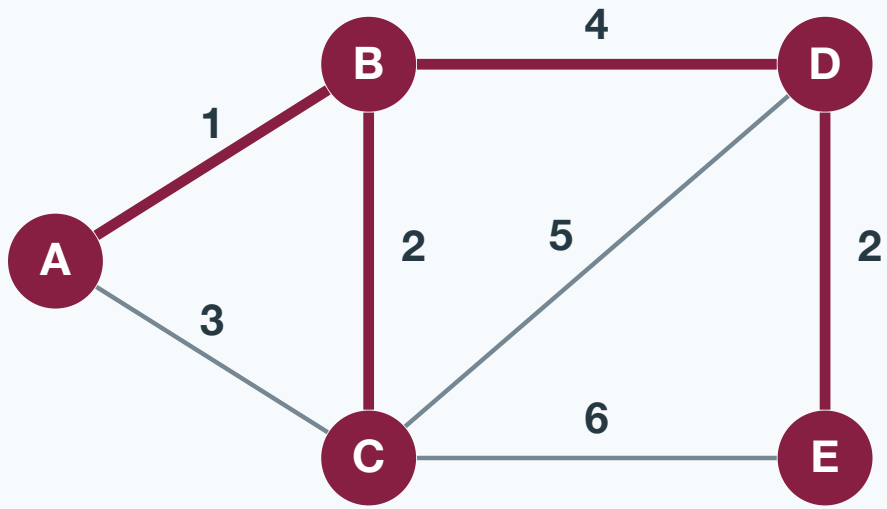
Undirected edges; labels are costs. Bold maroon edges have been selected.

Your turn: does this tree give every shortest route?

Work for about 4 minutes. The highlighted tree has minimum total edge cost 9.

1. What is the cost of the tree route from C to E?
2. Is that a shortest C-to-E route in the original graph?
3. Explain why an MST need not give shortest routes between every pair of nodes.

Compare the cost of a route with the sum of all selected tree-edge costs.



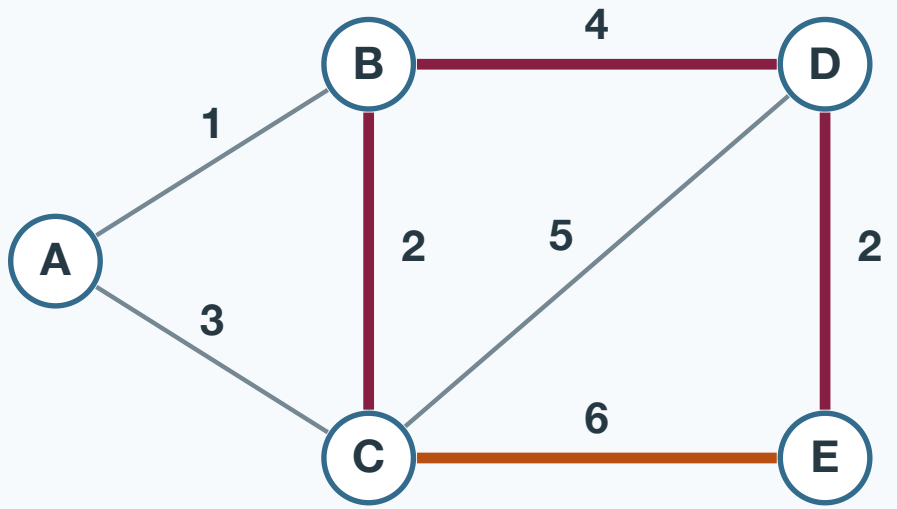
Undirected edges; labels are costs. Bold maroon edges have been selected.

Minimum tree cost and minimum route cost differ

$$C \to B \to D \to E: \quad 2 + 4 + 2 = 8.$$

The original graph also has the direct edge C–E of cost 6, so the tree route is not a shortest route.

> An MST minimizes the sum of all selected edge costs. A shortest-path problem minimizes the cost along one route. These are different objectives.



Maroon: the route in the MST, cost 8. Orange: the direct C–E route, cost 6.

Why cheapest crossing edges are safe

Cut property: in a connected undirected graph, a cheapest edge across a cut belongs to at least one MST. If it is uniquely cheapest, it belongs to every MST.

An optimal tree omitting that edge must use another crossing edge to connect the two groups. Exchange a suitable crossing edge for the cheapest one; the cost cannot increase.

Cycle property: an edge strictly more expensive than every other edge on a cycle belongs to no MST. With tied costs, do not conclude that every tied edge is excluded.

Appendix: complete exchange arguments and the condition needed for repeated choices.

Four algorithms, two different optimization goals

Algorithm	Objective	Selection/update
Dijkstra	Shortest routes; nonnegative costs	Smallest temporary distance
Bellman–Ford	Shortest routes; allows negative costs	Relax all arcs on each pass
Prim	Minimum total tree-edge cost	Cheapest edge crossing the reached-node cut
Kruskal	Minimum total tree-edge cost	Cheapest next edge that creates no cycle

> Practice naming the objective, checking the assumptions, and recording each algorithm state. A small label is meaningful only after we say what it measures.

Read BT §§7.9–7.10. Appendix: proofs, stopping conditions, and LP connections.

Appendix: why the algorithms work

These pages supply the complete arguments behind the main examples.

- Dijkstra finalization
- Bellman–Ford recurrence and stopping
- Detecting reachable negative cycles
- Shortest paths and the flow LP
- Why a tree has n – 1 edges
- Cut, cycle, Prim, and Kruskal proofs

Dijkstra: prove that a selected label is final

Assume all edge costs are nonnegative. Inductively, every previously finalized node has its true shortest distance. Every finite temporary label is the cost of a discovered route.

Let u be the next selected node. Suppose a shorter route to u existed. On that route, let v be the first unfinalized node and w its finalized predecessor. When w was finalized, relaxing w→v gave d(v) no larger than the cost of the route's prefix to v.

$$d(u) \le d(v) \le \text{prefix cost to } v \le \text{full competing route cost to } u.$$

> The first inequality follows from selecting the smallest temporary label. The last follows from nonnegative remaining costs. This contradicts a competing route costing less than d(u).

The base case is s, with distance 0. Induction proves every finalized label is correct.

Bellman–Ford: prove the recurrence

For $k = 0$, only the empty walk at s is allowed. Suppose row $k - 1$ gives the minimum costs using at most $k - 1$ arcs.

Any walk to v with at most k arcs either already uses at most $k - 1$ arcs, or ends with an arc (u, v). Removing that last arc leaves a walk to u with at most $k - 1$ arcs.

$$D_k(v) = \min\{D_{k-1}(v),\ \min_{(u,v)\in E}(D_{k-1}(u) + c_{uv})\}.$$

> Every candidate in this minimum is attained by an allowed walk whenever finite. Every allowed walk has cost at least one of these candidate minima. The formula is therefore exact.

If no negative cycle is reachable from s, removing cycles cannot increase route cost. A shortest route can be simple and uses at most n – 1 arcs.

Why improvement on pass n detects a negative cycle

If $D_n(v) < D_{n-1}(v)$, a walk attaining $D_n(v)$ must use n arcs. A walk with n arcs visits $n + 1$ node positions, so some node repeats.

If every cycle on that walk had nonnegative cost, deleting a cycle would give a walk of at most n – 1 arcs costing no more. This contradicts the strict improvement. Thus a negative cycle is reachable from s and can reach v.

> Conversely, if no label changes on a full pass, every reachable arc satisfies $d(v) \leq d(u) + c_{uv}$. Summing these inequalities around a reachable directed cycle implies its total cost is nonnegative.

Thus a reachable negative cycle forces some strict improvement on pass n. To find every affected node, start from improved nodes and follow directed arcs forward.

Why a shortest route solves the one-unit flow LP

Assume t is reachable from s and there is no negative-cost directed cycle. Decompose a feasible one-unit flow into path amounts $\alpha_P \geq 0$ and cycle amounts $\beta_C \geq 0$, with $\sum_P \alpha_P = 1$. Here P is an s-to-t path and C a directed cycle.

Cycle contributions have nonnegative cost. Each path has cost at least L, the shortest s-to-t path cost. Therefore every feasible flow has cost at least L.

$$\sum_P \alpha_P \, \mathrm{cost}(P) + \sum_C \beta_C \, \mathrm{cost}(C) \;\geq\; L \sum_P \alpha_P = L.$$

> Sending one unit along a shortest path attains L, proving equality of the two optima.

This proves that an optimal path flow exists, not that every optimal flow is a single path. A negative cycle anywhere in the uncapacitated LP would allow an unbounded circulation, even if it is unreachable from s.

Why a tree on n nodes has n – 1 edges

For n = 1, the tree has no edges. For n ≥ 2, take a longest simple path. An endpoint has only one neighbor: any extra neighbor would extend the path or create a cycle. Such a node is called a leaf.

Remove that leaf and its single incident edge. The remaining graph is connected and has no cycle, so it is a tree on n – 1 nodes.

$$|E(T)| = |E(T \setminus \{\text{leaf}\})| + 1 = (n-2) + 1 = n - 1.$$

> Adding an edge to a tree connects nodes already joined by one unique path, creating exactly one cycle. Deleting an edge on that cycle gives another spanning tree.

Conversely, any connected graph with n – 1 edges is a tree: a cycle edge could be removed without disconnecting it, contradicting the minimum n – 1 edges needed for connectivity.

The cut exchange argument preserves earlier choices

Let F be selected edges contained in some MST T. Choose a cut that no edge of F crosses. Let e be a cheapest edge crossing that cut.

If T already contains e, nothing is needed. Otherwise add e to T, creating a unique cycle. The path in T between the endpoints of e must cross the cut at an edge f.

$$c_e \le c_f, \qquad T' = T + e - f, \qquad \operatorname{cost}(T') \le \operatorname{cost}(T).$$

> Because no selected edge crosses the cut, f is not in F. Thus T′ is an MST containing every earlier choice and e. This is the repeated-choice form of the cut property.

For a uniquely cheapest e, any MST omitting e would strictly improve under the exchange. Hence every MST contains e.

The cycle exchange argument

Let e be the unique most expensive edge on an undirected cycle. Suppose an MST T contains e. Removing e from T creates two components.

The rest of the original cycle contains an edge f crossing between those components. That edge is cheaper than e. Replacing e by f reconnects T with no cycle and strictly lowers its cost, a contradiction.

> If e only ties for the maximum, the replacement may preserve cost. Then there exists an MST omitting e; it is incorrect to conclude that every MST omits e.

Kruskal’s rejection has a simpler immediate reason: the endpoints are already connected by selected edges. Adding the current edge would violate the requirement that selected edges have no cycle.

Why Prim and Kruskal finish at an MST

Prim: let F be the current tree. Its reached-node cut has no selected edge crossing it. The next cheapest crossing edge is safe by the cut exchange argument.

Kruskal: when an edge joins two selected components, take the cut around one component. Any cheaper crossing edge would already have been processed and would have joined components, so the accepted edge is cheapest across that cut.

Starting from the empty set, induction keeps the chosen edges inside at least one MST. When n – 1 edges have been selected without a cycle, they form a spanning tree and hence an MST.

Simple implementations: Prim $O(n^2 + m)$ by scans; Kruskal $O(m \log m)$ to sort, with efficient component tracking. Both permit negative edge costs: their proofs compare edges, not cumulative distances.